

Informe generacion de numeros aleatorios

Sebastian Bravo - 20241020004

Jose Cucanchon - 20241020038

La clase 'NumerosPseudoaleatorios' implementa un generador de números pseudoaleatorios basado en el algoritmo generador congruencial lineal. Para el usuario el proceso es muy sencillo: Escoger los límites inclusivos tanto inferior como superior, seleccionar cuantas veces desea que aparezca cada número y por último ingresar una semilla.

Ejemplo de la consola:

```
=== GENERADOR DE NÚMEROS ALEATORIOS UNIFORMES (LCG) ===
Ingrese el límite inferior del rango (ej. 0): 0
Ingrese el límite superior del rango (ej. 9): 10
Ingrese la cantidad de veces que debe salir CADA número: 2
Ingrese una semilla inicial (número entero positivo): 5
```

Output del ejemplo:

```
--- Secuencia Generada ---
[5, 8, 0, 3, 6, 9, 1, 4, 7, 10, 2, 5, 8, 0, 3, 6, 9, 1, 4, 7, 10, 2]
Total de números generados: 22
```

generarUniforme():

Se encarga de generar la lista final con los numeros que se generaron. Esta funcion necesita que se definan los limites inferior y superior del rango de numeros, el numero de repeticiones de cada numero y una semilla. Su función principal es determinar el tamaño del rango (el módulo) y, utilizando las otras dos funciones para obtener un multiplicador y un incremento, ejecuta un proceso iterativo que calcula y guarda cada número aleatorio de forma secuencial asegurando que aparezcan las veces indicadas.

```
public static List<Integer> generarUniforme(int min, int max, int repeticiones, Long semilla) {
    int m = (max - min) + 1; // Tamaño del rango

    // Selección de parámetros a y c según el Teorema de Hull-Dobell
    long a = calcularMultiplicador(m);
    long c = calcularIncremento(m);

    List<Integer> listaFinal = new ArrayList<>();
    long x = Math.abs(semilla) % m; // Ajustar la semilla al rango [0, m-1]

    // Repetimos los ciclos completos N veces para asegurar la distribución idéntica
    for (int r = 0; r < repeticiones; r++) {
        for (int i = 0; i < m; i++) {
            listaFinal.add((int) (x + min));
            x = (a * x + c) % m; // Fórmula LCG
        }
    }

    return listaFinal;
}
```

calcularIncremento():

Tiene como finalidad encontrar el valor del "incremento" (la constante c que se suma en la fórmula de generarUniforme()). Para que el generador funcione correctamente y recorra todos los números sin saltarse ninguno, esta función busca un número que sea "coprimo" con el tamaño del rango; es decir, hace pruebas hasta encontrar un valor cuyo único divisor en común con el tamaño del rango sea el número 1.

```
private static long calcularIncremento(int m) {
    for (long c = 3; c < m + 10; c += 2) {
        if (gcd(c, m) == 1) {
            return c;
        }
    }
    return 1;
}
```

calcularMultiplicador()

Se encarga de definir el "multiplicador" (la constante a de generarUniforme()). Para calcularlo, descompone el tamaño del rango en sus factores primos y construye un número a partir de ellos cumpliendo reglas matemáticas estrictas (el teorema de Hull-Dobell). En términos prácticos, asegura que la distancia matemática de cada salto garantice que la secuencia nunca se repita antes de completar todo el ciclo.

```
private static long calcularMultiplicador(int m) {
    List<Integer> factoresPrimos = obtenerFactoresPrimos(m);
    long productoPrimos = 1;
    for (int p : factoresPrimos) {
        productoPrimos *= p;
    }

    long k = productoPrimos;
    if (m % 4 == 0 && k % 4 != 0) {
        k *= 2;
    }

    long a = k + 1;
    // Evitamos un valor trivial de a = 1 si m es primo
    if (a == 2 && m > 2) {
        a += k;
    }
    return a;
}
```